Métodos de Optimización

Clase 4: Métodos basados en dirección de búsqueda

[Nombre del profesor]

[Nombre de la universidad / programa]

Contenido

Motivación y panorama general
Minimización 1D vía modelo cúbico
Descenso de gradiente
Minimización en una dirección (line search)
Descenso de gradiente con convergencia global
Método de Powell
Método de Newton
Newton con Hessiana aproximada (quasi-Newton, BFGS)
Trust region: método dogleg
Levenberg-Marquardt
Gradiente estocástico (SGD)
Síntesis

Motivación y panorama general

De dónde venimos

- **Clase 1**: conceptos analíticos — gradiente, Hessiana, convexidad, condiciones de optimalidad (KKT).
- **Clase 2**: métodos de búsqueda 1D/multidimensionales básicos — bisección, búsqueda áurea, grid search, random walk, Nelder-Mead.
- **Clase 3**: metaheurísticas estocásticas — basin hopping, simulated annealing, algoritmos genéticos, PSO.
- **Hoy (Clase 4)**: métodos *basados en dirección de búsqueda*, que usan explícitamente el gradiente (y a veces la Hessiana) para decidir **hacia dónde** moverse y **cuánto** avanzar.

Idea central de hoy

Todo método de esta familia repite: **(1)** elegir una dirección de descenso d_k, **(2)** decidir el tamaño del paso, **(3)** actualizar $x_{k+1} = x_k + \alpha_k d_k$. La diferencia entre métodos está en cómo se responde (1) y (2).

Dos familias de estrategias

Line search

1. Elegir primero la **dirección** d_k (ej. $-\nabla f$, Newton, quasi-Newton).
2. Buscar después **cuánto avanzar** α_k a lo largo de d_k.

Ejemplos de hoy: descenso de gradiente, Newton, quasi-Newton (BFGS), Powell.

Trust region

1. Fijar una **región de confianza** (radio Δ_k) alrededor de x_k.
2. Resolver un subproblema restringido: dirección **y** magnitud del paso a la vez.

Ejemplo de hoy: trust region dogleg, Levenberg-Marquardt.

Ambas familias garantizan, bajo hipótesis razonables, **convergencia global**: desde *cualquier* punto inicial, la sucesión x_k se acerca a un punto estacionario.

Funciones de prueba de hoy

Cuadrática mal condicionada

$$f(x, y) = \tfrac{1}{2}\left(x^2 + \kappa y^2\right), \quad \kappa = 25$$

Convexa, mínimo único en el origen. El número de condición κ de la Hessiana controla qué tan "alargadas" son las curvas de nivel $\rightarrow$ ilustra el zig-zag del descenso de gradiente.

Rosenbrock (función "banana")

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

No convexa, valle curvo muy estrecho, mínimo global en $(1, 1)$ con $f = 0$. Estándar en la literatura para comparar Newton, quasi-Newton, trust region y métodos sin derivadas.

En todos los códigos: parte **manual** (para entender el mecanismo) + comparación con la función equivalente de `scipy.optimize`.

Minimización 1D vía modelo cúbico

Por qué empezar por aquí

- Casi todos los métodos de hoy (line search) necesitan resolver, **muchas veces por iteración**, un subproblema de una sola variable:

$$\min_{\alpha \geq 0} \ \varphi(\alpha) = f(x_k + \alpha\, d_k)$$

- En la Clase 2 vimos bisección y búsqueda áurea: solo usan **evaluaciones** de φ, convergen **linealmente**.
- Pero en optimización **siempre** tenemos $\varphi'(\alpha)$ disponible gratis: $\varphi'(\alpha) = \nabla f(x_k + \alpha d_k)^T d_k$.

Idea de hoy

Usar φ **y** φ' en dos puntos para construir un **modelo cúbico** de φ, y minimizar ese modelo en forma cerrada — exactamente el mismo espíritu que usará Newton más adelante con un modelo cuadrático en varias variables, pero aquí en 1D y con un cúbico (para capturar la información de derivada en *dos* puntos a la vez).

El modelo cúbico

Dados dos puntos a, b con $\varphi(a), \varphi'(a), \varphi(b), \varphi'(b)$ conocidos, existe un único polinomio cúbico

$$p(\alpha) = A\alpha^3 + B\alpha^2 + C\alpha + D$$

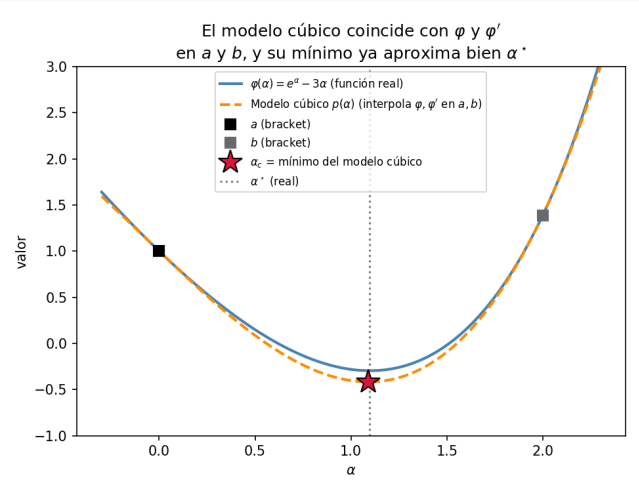
que coincide con φ **y** φ' en a y en b (4 condiciones, 4 coeficientes — sistema lineal 4×4).

Si el intervalo encierra un mínimo
Si $\varphi'(a) < 0 < \varphi'(b)$ (bracket válido), el cúbico p tiene un mínimo local **dentro** de $[a, b]$, y ese mínimo tiene **fórmula cerrada** (Nocedal & Wright, ec. 3.59):

$$d_1 = \varphi'(a) + \varphi'(b) - 3\frac{\varphi(a) - \varphi(b)}{a - b}, \qquad d_2 = \operatorname{sgn}(b - a)\sqrt{d_1^2 - \varphi'(a)\varphi'(b)}$$

$$\alpha_c = b - (b - a)\,\frac{\varphi'(b) + d_2 - d_1}{\varphi'(b) - \varphi'(a) + 2d_2}$$

Visualización: el modelo cúbico junto a φ



Algoritmo: interpolación cúbica con bracket

Análogo a bisección, pero reemplazando el punto medio por el mínimo del modelo cúbico:

1. Partir de un bracket $[a, b]$ con $\varphi'(a) < 0 < \varphi'(b)$.
2. Calcular α_c con la fórmula cerrada del cúbico.
3. Evaluar $\varphi'(\alpha_c)$:
   - si tiene el mismo signo que $\varphi'(a)$: $a \leftarrow \alpha_c$.
   - si no: $b \leftarrow \alpha_c$.
4. Repetir hasta $|\varphi'(\alpha_c)| < \text{tol}$.

Por qué importa
Esta es, literalmente, la estrategia que usan internamente los line search *serios* (condiciones de Wolfe) para refinar α una vez que se tiene un bracket: converge **superlinealmente**, muchas menos evaluaciones que bisección o búsqueda áurea.

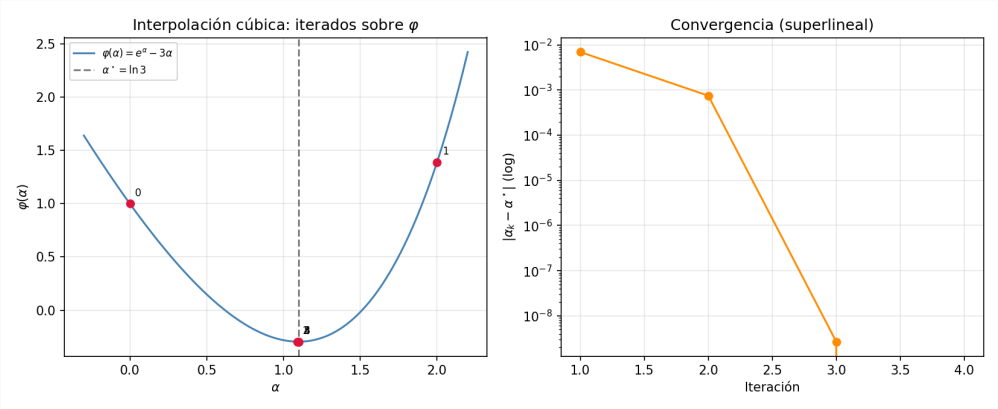
Código: minimización 1D por modelo cúbico

```
def cubic_step(a, b, phi, dphi):
    fa, fb = phi(a), phi(b);  ga, gb = dphi(a), dphi(b)
    d1 = ga + gb - 3.0 * (fa - fb) / (a - b)
    s = np.sign(b - a)
    d2 = s * np.sqrt(max(d1**2 - ga * gb, 0.0))
    return b - (b - a) * (gb + d2 - d1) / (gb - ga + 2 * d2)

def cubic_interpolation_1d(phi, dphi, a, b, tol=1e-10, max_iter=30):
    for _ in range(max_iter):
        alpha_c = cubic_step(a, b, phi, dphi)
        g_c = dphi(alpha_c)
        if abs(g_c) < tol:
            break
        if np.sign(g_c) == np.sign(dphi(a)):
            a = alpha_c
        else:
            b = alpha_c
    return alpha_c
```

Ejemplo: $\varphi(\alpha) = e^{\alpha} - 3\alpha$

Mínimo exacto: $\varphi'(\alpha) = e^{\alpha} - 3 = 0 \Rightarrow \alpha^{\star} = \ln 3 \approx 1{,}0986122887$. Bracket inicial $[0, 2]$ $(\varphi'(0) = -2 < 0,\ \varphi'(2) = e^2 - 3 > 0)$.



Resultados

- **Interpolación cúbica**: **4** iteraciones, $\alpha_{\text{final}} = 1{,}0986122887$ — coincide con $\alpha^\star$ hasta precisión de máquina.
- `scipy.optimize.minimize_scalar` (Brent, **sin** derivadas): 10 iteraciones para la misma precisión.

La lección que llevamos al resto de la clase

Usar **derivadas** (aquí, φ'; más adelante, el gradiente completo ∇f y hasta la Hessiana H) para construir un **modelo local** y minimizar ese modelo en vez de la función real, es la idea que atraviesa *todos* los métodos de esta clase — del modelo lineal (descenso de gradiente) al cuadrático (Newton, trust region) pasando por este modelo cúbico en 1D.

Descenso de gradiente

Idea: el gradiente como dirección de máximo descenso

Del modelo lineal local de Taylor:

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^T p$$

La dirección p (de norma fija) que **minimiza** este modelo lineal es

$$d_k = -\nabla f(x_k)$$

(recordar Clase 1: Cauchy-Schwarz $\Rightarrow -\nabla f$ es la dirección de máximo decrecimiento local).

Algoritmo: descenso de gradiente

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Repetir hasta $\|\nabla f(x_k)\| < \text{tol}$.

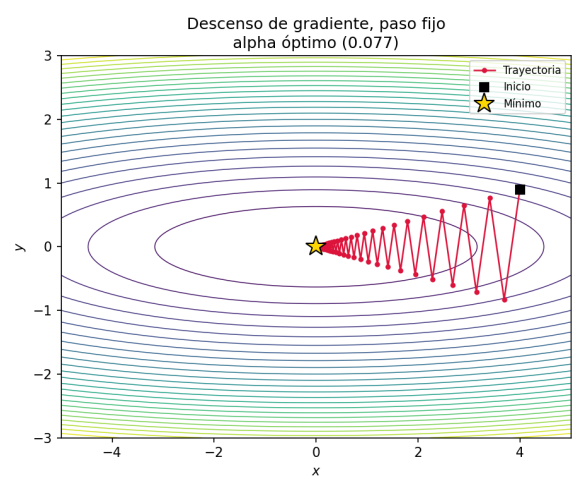
La pregunta clave: **cómo elegir** α_k. Hoy vemos dos respuestas: paso fijo (ingenuo) y paso variable con line search (robusto).

Código: descenso de gradiente, paso fijo

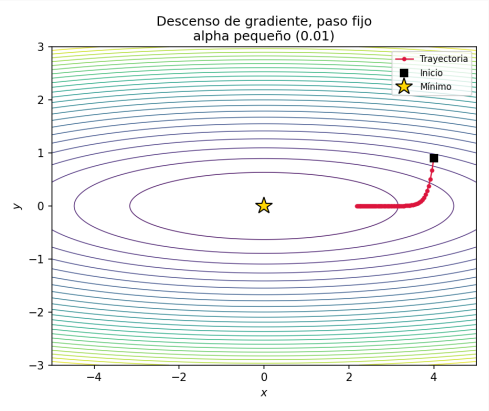
```python
def gradient_descent_fixed(f, grad, x0, alpha, n_iter=40):
    x = np.array(x0, dtype=float)
    traj = [x.copy()]
    for _ in range(n_iter):
        x = x - alpha * grad(x)
        traj.append(x.copy())
    return np.array(traj)
```

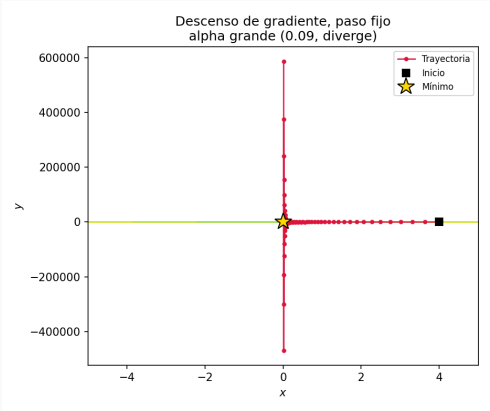
Extremadamente simple — y extremadamente sensible a la elección de `alpha`, como veremos.

Trayectoria: efecto del tamaño de paso fijo



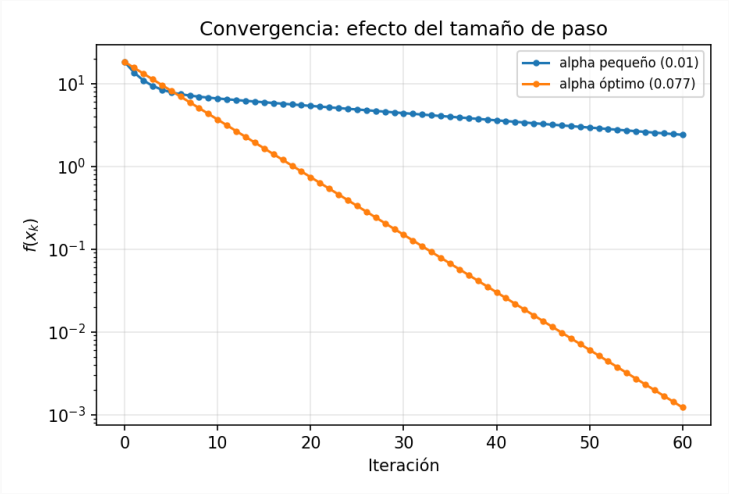
Comparación: paso pequeño vs. paso óptimo vs. divergencia





- Paso teórico óptimo (autovalores $\{1, \kappa\}$): $\alpha^\star = 2/(1+\kappa)$.
- Con $\alpha > 2/\lambda_{\text{máx}}$ el método **diverge**: $\alpha = 0{,}09$ produce $f_{\text{final}} \approx 4 \times 10^{12}$ tras 60 iteraciones.

Convergencia: paso pequeño vs. óptimo



El problema de fondo: elegir α a mano no escala

- El paso óptimo $\alpha^\star = 2/(1+\kappa)$ solo existe en forma cerrada porque f es **cuadrática**. Para una f general, no sabemos κ de antemano.
- Un α fijo que funciona en una región de f puede divergir en otra (curvatura variable).
- **Solución**: en lugar de fijar α a mano, calcularlo *en cada iteración* resolviendo (aprox.) un problema de minimización 1D. Eso es **line search** — el tema que sigue.

Minimización en una dirección (line search)

Formalización: la función de mérito $\varphi(\alpha)$

Dado x_k y una dirección de descenso d_k (es decir, $\nabla f(x_k)^T d_k < 0$), se define la función de una variable

$$\varphi(\alpha) = f(x_k + \alpha d_k), \qquad \alpha \geq 0$$

Line search exacto: resolver $\min_{\alpha \geq 0} \varphi(\alpha)$ (ej. con Brent 1D). Preciso, pero caro: requiere muchas evaluaciones de f para un solo paso de x_k.

Line search inexacto: en vez de minimizar φ exactamente, aceptar el primer α que dé una reducción "suficiente". Mucho más barato y es lo que se usa en la práctica.

Condición de Armijo (descenso suficiente)

Condición de Armijo

$$\varphi(\alpha) \le \varphi(0) + c_1\,\alpha\,\varphi'(0), \qquad c_1 \in (0,1) \text{ (típicamente } c_1 = 10^{-4})$$

Geométricamente: la recta $\varphi(0) + c_1\alpha\varphi'(0)$ tiene la pendiente inicial de φ "achicada" por c_1; cualquier α cuya $\varphi(\alpha)$ quede **por debajo** de esa recta se acepta.

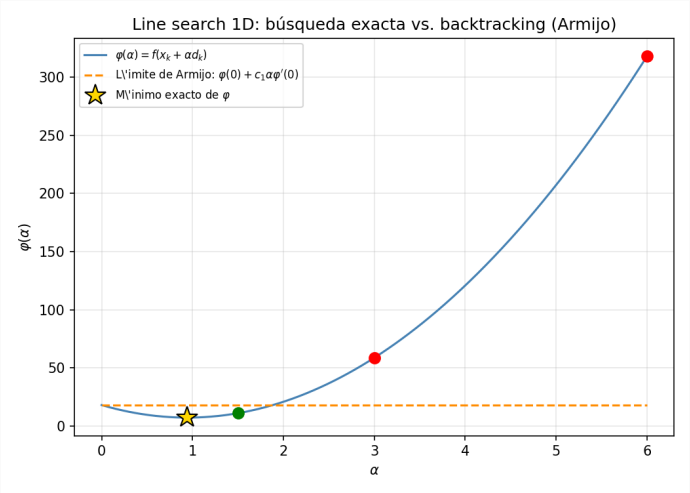
Backtracking (algoritmo práctico)

1. Empezar con $\alpha = \alpha_0$ (ej. $\alpha_0 = 1$).
2. Si se cumple Armijo, aceptar α y terminar.
3. Si no, $\alpha \leftarrow \rho\,\alpha$ (ej. $\rho = 0{,}5$) y volver a 2.

Código: backtracking Armijo

```
def armijo_backtracking(f, grad, x, d, alpha0=1.0, c1=1e-4, rho=0.5):
    phi0 = f(x)
    dphi0 = grad(x) @ d
    alpha = alpha0
    while f(x + alpha * d) > phi0 + c1 * alpha * dphi0:
        alpha *= rho
    return alpha
```

Visualización: $\varphi(\alpha)$, límite de Armijo y backtracking



Lectura de la figura

- El mínimo **exacto** de φ (estrella dorada) está en $\alpha^\star \approx 0{,}94$.
- Backtracking arranca en $\alpha_0 = 6$ y prueba $6 \to 3 \to 1{,}5$: los dos primeros se **rechazan** (quedan por encima de la recta límite de Armijo), el tercero se **acepta**.
- El α aceptado por Armijo (1.5) no es el mínimo exacto de φ — y no hace falta que lo sea: solo se pide una reducción suficiente, mucho más barato de verificar.

Nota

En la práctica se suele combinar Armijo con una segunda condición (curvatura, condiciones de Wolfe) para evitar pasos demasiado cortos. Aquí usamos solo Armijo + backtracking por simplicidad, suficiente para garantizar convergencia global junto con descenso de gradiente.

Descenso de gradiente con convergencia global

Combinando gradiente + line search

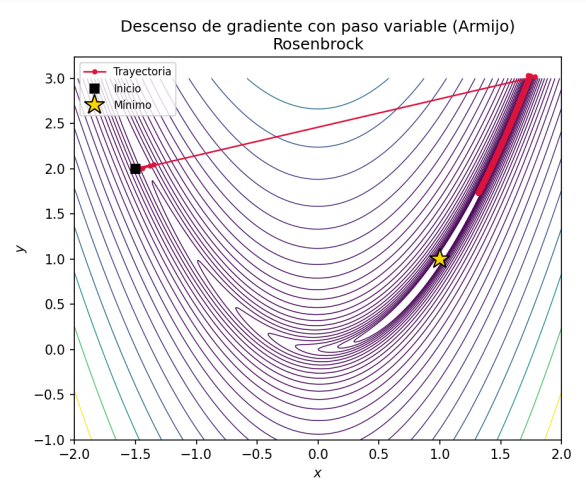
Algoritmo: descenso de gradiente con paso variable

1. $d_k = -\nabla f(x_k)$
2. $\alpha_k =$ backtracking Armijo a lo largo de d_k (line search 1D)
3. $x_{k+1} = x_k + \alpha_k d_k$
4. Repetir hasta $\|\nabla f(x_k)\| < \text{tol}$

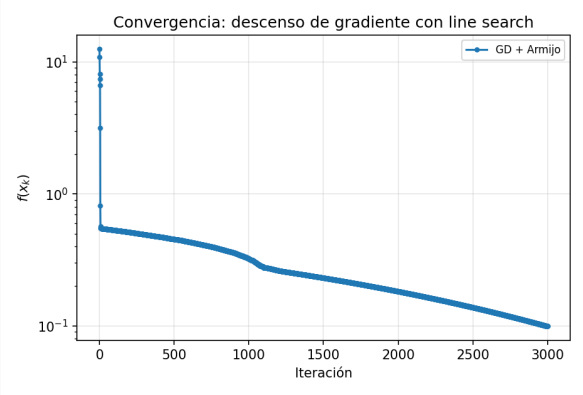
***Convergencia global* — precisión de lenguaje**
Significa que el método converge a un punto **estacionario** desde *cualquier* punto inicial — NO que encuentre el mínimo global de una función no convexa. Es terminología estándar (Nocedal & Wright, cap. 3), distinta de *óptimo global*.

A diferencia del paso fijo (script anterior), este método **nunca diverge**: Armijo garantiza que cada paso efectivamente reduce f.

Trayectoria en Rosenbrock



Convergencia — y su costo



- GD + Armijo: **no converge del todo** en 3000 iter. ($f \approx 0{,}099$).
- `scipy CG` (Wolfe): converge en solo **29** iter.

Punto pedagógico

Robusto $\neq$ rápido. El valle curvo de Rosenbrock es el peor caso para descenso de gradiente puro. Motiva usar **curvatura** (Newton/quasi-Newton).

Método de Powell

Motivación: optimización sin gradiente

- Todos los métodos anteriores necesitan ∇f (analítico o automático).
- ¿Qué hacer si f es una caja negra (ej. una simulación costosa) sin gradiente disponible?
- **Powell (1964)**: minimiza sucesivamente a lo largo de **direcciones**, usando solo evaluaciones de f (line search 1D repetido), y construye **direcciones conjugadas** sobre la marcha para acelerar la convergencia respecto a usar siempre los ejes coordenados.

Direcciones conjugadas (recordatorio, cuadráticas)
Dos direcciones d_i, d_j son conjugadas respecto a H si $d_i^T H d_j = 0$. Minimizar sucesivamente a lo largo de n direcciones conjugadas resuelve una cuadrática n-dimensional **exactamente** en n pasos — mucho mejor que los ejes coordenados, que hacen zig-zag igual que el descenso de gradiente.

Algoritmo de Powell (un ciclo, $n = 2$)

Direcciones iniciales: $d_1 = e_1 = (1, 0)$, $d_2 = e_2 = (0, 1)$.

1. Desde x_0: line search exacto a lo largo de $d_1 \to x_1$.
2. Desde x_1: line search exacto a lo largo de $d_2 \to x_2$.
3. Nueva dirección conjugada: $d_{\text{new}} = x_2 - x_0$.
4. Line search a lo largo de d_{new} desde $x_2 \to x_3$.
5. **Regla de Powell**: descartar la dirección que más redujo f en este ciclo, reemplazarla por d_{new}.
6. Repetir el ciclo con el conjunto de direcciones actualizado.

Salvaguarda usada en el código

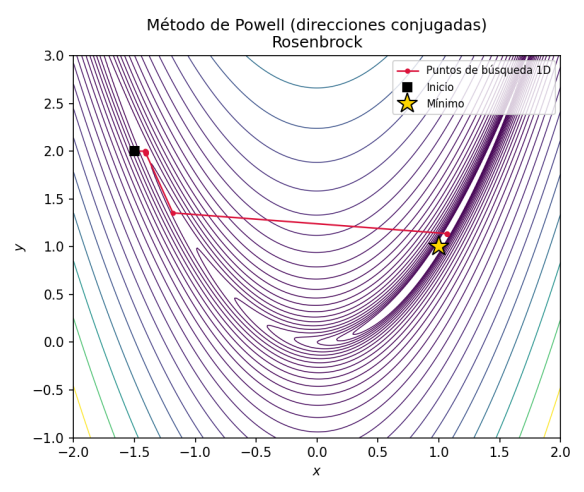
En valles muy curvos (Rosenbrock), un line search 1D puede no ser unimodal en el rango buscado. Se agrega la salvaguarda: si el resultado es peor que no moverse, se conserva el punto de partida.

Código: núcleo del método de Powell

```
def line_search_along(f, x, d, bracket=(-2, 2)):
    def phi(alpha):
        return f(x + alpha * d)
    res = minimize_scalar(phi, bracket=bracket)
    if res.fun <= phi(0.0):
        return x + res.x * d, res.fun
    return x.copy(), phi(0.0)   # salvaguarda

# ciclo: line search a lo largo de cada direccion,
# luego direccion conjugada d_new = x_final - x_inicial,
# reemplazar la direccion que mas redujo f por d_new
```

Trayectoria: Powell en Rosenbrock



Resultados y comparación con `scipy`

- **Powell manual**: 5 ciclos (15 line searches 1D), $f_{\text{final}} \approx 4{,}4 \times 10^{-3}$.
- `scipy.optimize.minimize(method='Powell')`: 25 iteraciones, 670 evaluaciones de f, $f_{\text{final}} \approx 2{,}7 \times 10^{-29}$ (esencialmente exacto).

Por qué la diferencia

La implementación de `scipy` incluye salvaguardas adicionales (verificación de que las direcciones no se vuelvan casi paralelas/linealmente dependientes, reinicios periódicos a los ejes coordenados) que nuestra versión didáctica omite por claridad. El mecanismo de fondo — direcciones conjugadas construidas sin gradiente — es el mismo.

Método de Newton

Del modelo lineal al modelo cuadrático

Descenso de gradiente usa el modelo **lineal** de Taylor. El método de Newton usa el modelo **cuadrático** (Clase 1, serie de Taylor de 2do orden):

$$m_k(p) = f(x_k) + \nabla f(x_k)^T p + \tfrac{1}{2}\, p^T H(x_k)\, p$$

Minimizando m_k respecto a p (asumiendo $H(x_k)$ definida positiva):

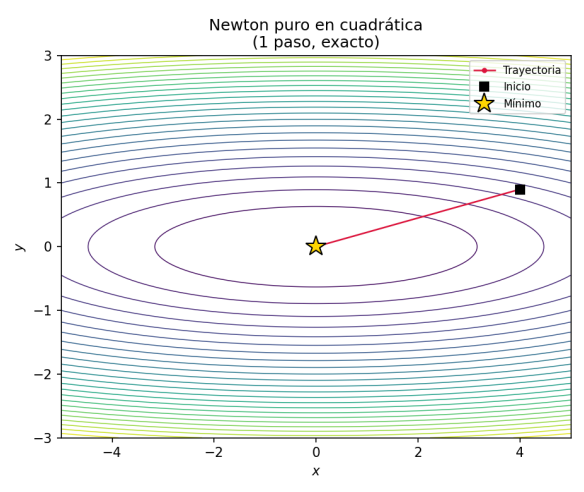
$$\nabla_p m_k(p) = \nabla f(x_k) + H(x_k)\, p = 0 \implies p_k = -H(x_k)^{-1}\, \nabla f(x_k)$$

Paso de Newton puro

$$x_{k+1} = x_k + p_k \qquad (\alpha_k = 1 \text{ fijo})$$

En una cuadrática exacta, el modelo m_k coincide con $f \Rightarrow$ **un solo paso** basta.

Trayectoria: Newton puro en la cuadrática



Newton en funciones no cuadráticas: convergencia y riesgos

- Para f general, m_k solo aproxima a f localmente $\Rightarrow$ Newton necesita varias iteraciones, pero converge con **orden cuadrático** cerca del mínimo (el número de dígitos correctos se duplica en cada paso).
- **Riesgo**: si $H(x_k)$ no es definida positiva (lejos del mínimo, en zonas de curvatura mixta), p_k puede no ser dirección de descenso, o incluso apuntar hacia un máximo/silla.

Newton amortiguado (*damped Newton*)
Salvaguarda práctica:

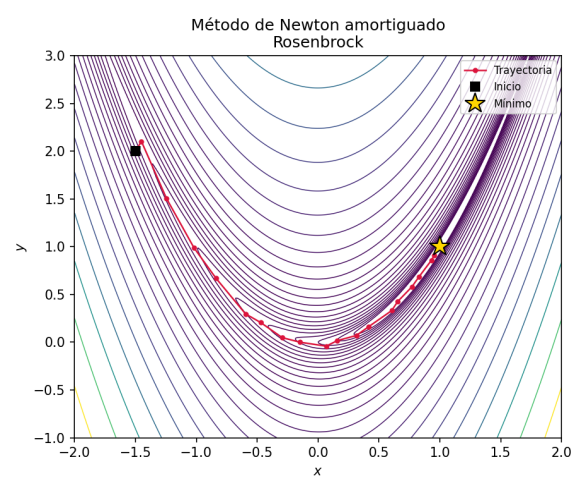
1. Si $\nabla f(x_k)^T p_k \geq 0$ (no es descenso), usar $p_k = -\nabla f(x_k)$ en su lugar.
2. Aplicar backtracking Armijo sobre α (en vez de $\alpha = 1$ fijo).

Así se preserva la velocidad de Newton cerca del óptimo, con la robustez de line search lejos de él.

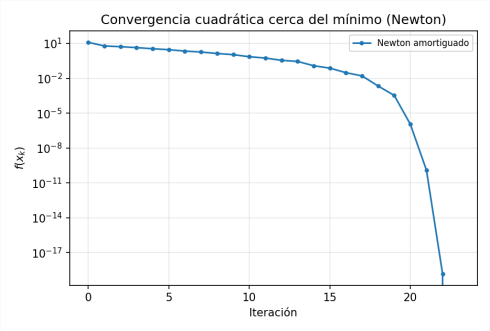
Código: Newton amortiguado

```
g = grad(x); H = hess(x)
p = np.linalg.solve(H, -g)          # paso de Newton
if g @ p >= 0:                       # salvaguarda: no es descenso
    p = -g
alpha = 1.0
while f(x + alpha*p) > f(x) + 1e-4*alpha*(g @ p):
    alpha *= 0.5                     # backtracking Armijo
x = x + alpha * p
```

Trayectoria: Newton amortiguado en Rosenbrock



Resultados: velocidad de Newton



- Newton amortiguado: **24** iteraciones hasta $f \approx 0$.
- `scipy Newton-CG`: 191 iteraciones (usa un solver iterativo –gradiente conjugado– para el sistema lineal, en vez de resolverlo exactamente como aquí).
- Comparar con descenso de gradiente + Armijo: **3000** iteraciones sin convergencia completa (script anterior). La curvatura (Hessiana) es lo que marca la diferencia.

Newton con Hessiana aproximada (quasi-Newton, BFGS)

Motivación

- Newton exacto necesita calcular **y factorizar** $H(x_k)$ en cada iteración: costoso ($O(n^3)$ para factorizar, $O(n^2)$ para almacenar), y a veces H ni siquiera está disponible analíticamente.
- **Idea quasi-Newton**: construir una **aproximación** $B_k \approx H(x_k)$ (o directamente de su inversa) usando solo información de **gradientes** ya calculados — sin nunca evaluar la Hessiana real.

BFGS (Broyden–Fletcher–Goldfarb–Shanno)
Con $s_k = x_{k+1} - x_k$ y $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$:

$$H_{k+1}^{-1} = (I - \rho_k s_k y_k^T)\, H_k^{-1}\, (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}$$

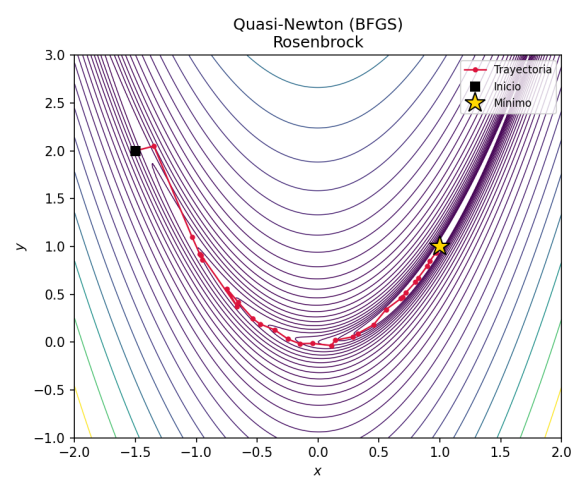
Propiedades clave de BFGS

- Se inicializa $H_0^{-1} = I$ (el primer paso equivale a descenso de gradiente).
- Cada iteración: $p_k = -H_k^{-1}\nabla f(x_k)$, luego line search (Armijo) en esa dirección.
- **Condición de curvatura**: si $y_k^T s_k > 0$, la actualización preserva automáticamente que H_k^{-1} sea definida positiva $\Rightarrow$ p_k **siempre** es dirección de descenso (a diferencia de Newton puro, no hace falta verificar autovalores).
- Convergencia **superlineal** (más lenta que la cuadrática de Newton, pero sin necesitar la Hessiana real).

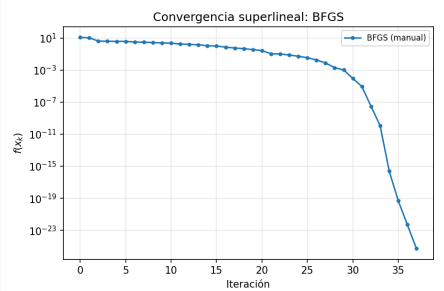
Código: actualización BFGS

```
s = x_new - x;  y = g_new - g
sy = y @ s
if sy > 1e-12:                          # condicion de curvatura
    rho = 1.0 / sy
    V = np.eye(n) - rho * np.outer(s, y)
    Hinv = V @ Hinv @ V.T + rho * np.outer(s, s)
# si sy <= 0, se conserva Hinv (salvaguarda)
```

Trayectoria: BFGS en Rosenbrock



Resultados



- BFGS manual: **38** iteraciones, $f_{\text{final}} \approx 5 \times 10^{-26}$.
- `scipy.optimize.minimize(method='BFGS')`: 37 iteraciones — prácticamente idéntico.
- Tabla comparativa (misma Rosenbrock, mismo punto inicial): Newton exacto 24 iter, BFGS 38 iter, descenso de gradiente+Armijo no converge en 3000.

Trust region: método dogleg

Trust region: otra forma de usar el modelo cuadrático

Recordar el modelo cuadrático local:

$$m_k(p) = f(x_k) + \nabla f(x_k)^T p + \tfrac{1}{2}\, p^T H_k\, p$$

Line search (Newton, BFGS): confiar en m_k en *toda* una dirección, y ajustar solo cuánto avanzar.

Trust region: reconocer que m_k solo es confiable **cerca** de x_k, y resolver:

$$\min_p \; m_k(p) \qquad \text{sujeto a} \qquad \|p\| \le \Delta_k$$

Esto decide **dirección y magnitud del paso simultáneamente**, dentro de una bola de radio Δ_k (la "región de confianza").

El subproblema exacto es caro
Resolver el subproblema restringido *exactamente* requiere álgebra no trivial. El método **dogleg** lo aproxima de forma barata y muy efectiva.

Trust region en 1D: el subproblema se resuelve a mano

Una variable: el modelo es una parábola y la región de confianza es un **intervalo**, con $g = f'(x_k)$, $h = f''(x_k)$:

$$\min_p \; m_k(p) = f(x_k) + g\,p + \tfrac{1}{2}\,h\,p^2 \quad \text{s.a.} \quad |p| \le \Delta_k$$

El término constante $f(x_k)$ no cambia dónde está el mínimo, pero sí es parte del modelo (la parábola naranja de las figuras pasa por $(x_k, f(x_k))$).

Tres casos

1. $h > 0$ y $|g/h| \le \Delta_k$: el mínimo de la parábola cabe $\Rightarrow$ **paso de Newton**, $p = -g/h$.
2. $h > 0$ y $|g/h| > \Delta_k$: queda fuera $\Rightarrow$ **al borde**, $p = -\mathrm{sgn}(g)\,\Delta_k$.
3. $h \le 0$ (curvatura negativa o nula): no hay mínimo $\Rightarrow$ **al borde** cuesta abajo, $p = -\mathrm{sgn}(g)\,\Delta_k$.

¿Y el paso de Cauchy y el dogleg en 1D?

- En 1D, $-g$ (dirección de Cauchy) y $-g/h$ (dirección de Newton) apuntan **al mismo lado**: son **colineales**.
- Por eso la poligonal dogleg se reduce a un segmento recto, y el paso es simplemente *Newton recortado a la región de confianza*.

Lo que sí se ve igual que en varias variables

El papel de la región de confianza (limitar el paso), el manejo de curvatura negativa (ir al borde) y la regla de aceptación y de radio con ρ_k. En 2D o más, Cauchy y Newton apuntan en direcciones distintas, y ahí sí hace falta combinarlos: es el dogleg de las siguientes diapositivas.

¿Se acepta el paso? El ratio ρ_k y el radio

El modelo puede equivocarse lejos de x_k. Se compara lo que **predijo** contra lo que **pasó**:

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)} = \frac{\text{reducción real}}{\text{reducción predicha}}$$

Aceptar o rechazar

- $\rho_k > 0$: f bajó $\Rightarrow$ se **acepta** $x_{k+1} = x_k + p_k$.
- $\rho_k \leq 0$: f no bajó $\Rightarrow$ se **rechaza**, $x_{k+1} = x_k$.

Qué hacer con Δ_k

- $\rho_k < 1/4$: **encoger**, $\Delta/4$.
- $\rho_k > 3/4$ y $|p_k| = \Delta_k$: **crecer**, 2Δ.
- en otro caso: **mantener**.

Es exactamente la misma regla que se usa en dogleg y Levenberg-Marquardt; en 1D se puede ver paso a paso.

¿Por qué basta $\rho_k > 0$?

Un cociente de dos negativos también es positivo: ¿se aceptaría un paso donde f y el modelo *suben*?

No: el denominador nunca es negativo
$p = 0$ es factible y el paso minimiza (aprox.) el modelo en la región, así que $m_k(p) \le m_k(0)$:

$$m_k(0) - m_k(p) \;\ge\; 0 \qquad (\text{y } > 0 \text{ si } g \neq 0)$$

Por tanto ρ_k tiene el signo de la reducción **real**: $\rho_k > 0 \iff f$ bajó.

Verificación en 1D, con pred $= -(g\,p + \frac{1}{2} h\,p^2)$:

Caso	Paso p	Reducción predicha
$h > 0$, Newton cabe	$-g/h$	$\dfrac{g^2}{2h} > 0$
$h > 0$, $\lvert g/h\rvert > \Delta_k$	$-\mathrm{sgn}(g)\Delta_k$	$\Delta_k\big(\lvert g\rvert - \frac{1}{2} h\Delta_k\big) > \frac{1}{2}\lvert g\rvert\Delta_k > 0$
$h \le 0$	$-\mathrm{sgn}(g)\Delta_k$	$\lvert g\rvert\Delta_k - \frac{1}{2} h\Delta_k^2 \ge \lvert g\rvert\Delta_k > 0$

Condición de descenso de Cauchy
Todo paso aceptable debe reducir el modelo al menos tanto como el punto de Cauchy:

$$m_k(0) - m_k(p_k) \;\geq\; \tfrac{1}{2}\,\|\nabla f_k\|\,\min\!\Big(\Delta_k,\; \frac{\|\nabla f_k\|}{\|H_k\|}\Big) > 0$$

Cauchy la cumple siempre. El dogleg la cumple si H_k es **definida positiva**: el modelo decrece a lo largo de la poligonal (Nocedal & Wright, Lema 4.2).

Si H_k no es definida positiva
p_N ya no minimiza el modelo (puede ser un punto silla) y este puede *subir*. Ejemplo: $g = (1, 1)$, $H = \mathrm{diag}(-1, 2)$, $\Delta = 10$ da $m(0) - m(p_N) = -0{,}25 < 0$. Por eso el código usa **solo Cauchy** si H_k no es definida positiva.

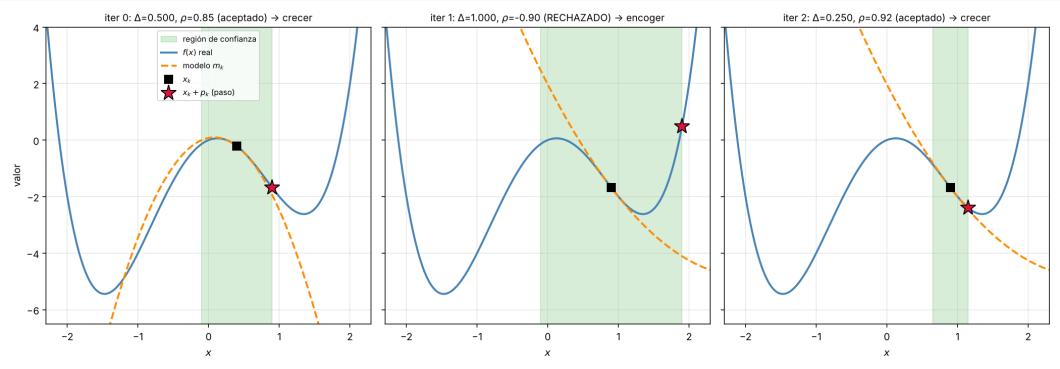
`rho = actual/pred if pred > 1e-14 else -1.0` también cubre pred ≈ 0 (ya en el mínimo): no se divide y se rechaza.

Código: trust region en 1D

```
def tr_step_1d(g, h, Delta):
    if h > 0 and abs(g / h) <= Delta:
        return -g / h                    # caso 1: Newton cabe
    return -np.sign(g) * Delta           # casos 2 y 3: al borde
while abs(df(x)) > tol:
    g, h = df(x), d2f(x)
    p = tr_step_1d(g, h, Delta)
    pred   = -(g * p + 0.5 * h * p**2)   # m(0) - m(p)
    actual = f(x) - f(x + p)
    rho = actual / pred if pred > 1e-14 else -1.0
    if rho < 0.25:
        Delta = 0.25 * Delta                     # encoger
    elif rho > 0.75 and np.isclose(abs(p), Delta):
        Delta = min(2 * Delta, Delta_max)        # crecer
    if rho > 0:
        x = x + p                        # aceptar; si no, x no cambia
```

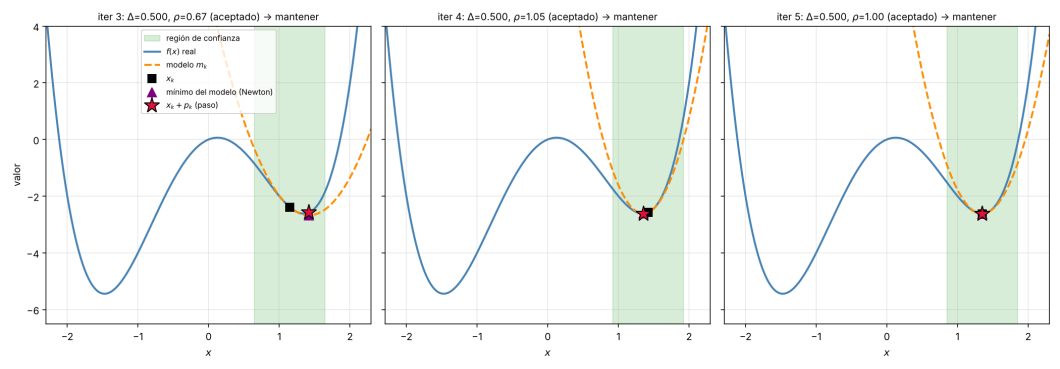
Ejemplo: $f(x) = x^4 - 4x^2 + x$, partiendo de $x_0 = 0{,}4$, $\Delta_0 = 0{,}5$

Dos mínimos locales (en $\approx -1{,}47$ y $\approx 1{,}35$) y un máximo local cerca de $x \approx 0{,}13$. En $x_0 = 0{,}4$: $f''(x_0) = -6{,}08 < 0$, **curvatura negativa**: Newton puro iría hacia el máximo.



Ejemplo (cont.): cerca del mínimo, el modelo ya es fiel

Con $\Delta = 0{,}5$ el mínimo de la parábola (triángulo morado) **cabe** en la región: el paso es el de Newton completo y $\rho_k \approx 1$.

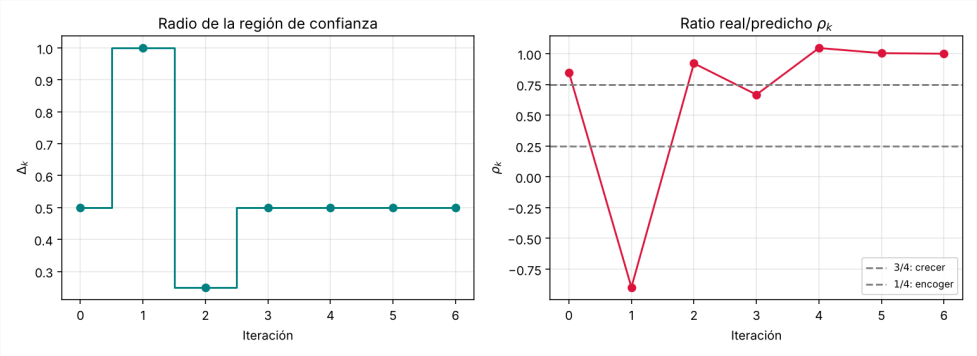


Lectura iteración por iteración

k	x_k	$f''(x_k)$	Δ_k	ρ_k	qué ocurre
0	0.4000	$-6{,}08$	0.500	0.85	curvatura < 0: va al borde; acepta; **crece**
1	0.9000	1,72	1.000	$-0{,}90$	Newton recortado; f **sube**: **rechaza** y encoge
2	0.9000	1,72	0.250	0.92	mismo x_k, radio menor; acepta; crece
3	1.1500	7,87	0.500	0.67	Newton cabe en la región; acepta
4	1.4189	16,16	0.500	1.05	Newton; acepta
5	1.3524	13,95	0.500	1.00	Newton; acepta
6	1.3470	13,77	0.500	1.00	Newton; $\lvert f' \rvert < 10^{-6}$, termina

- **Iter. 1**: con $\Delta = 1$ el modelo prometía bajar mucho, pero f sube ($\rho < 0$). Al achicar la región a 0,25 el modelo vuelve a ser fiel.
- **Cerca del mínimo** el modelo es casi exacto ($\rho \approx 1$), el radio ya no estorba y se recupera la convergencia rápida de Newton.

Radio Δ_k y ratio ρ_k a lo largo de las iteraciones



- Trust region 1D (manual): **7** iteraciones, $x^\star \approx 1{,}34699741$, $f^\star \approx -2{,}6186$.
- `scipy trust-exact`: mismo mínimo en 4 iteraciones (parámetros distintos: radio inicial, umbrales de aceptación y resolución del subproblema); el número exacto depende de esas elecciones.

Los dos candidatos que combina dogleg

Paso de Cauchy p_C
El minimizador de m_k a lo largo de la dirección de máximo descenso $-\nabla f(x_k)$, restringido a la bola:

$$p_C = -\frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T H_k \nabla f_k} \nabla f_k \quad \text{(truncado a la frontera si se sale de } \Delta_k)$$

Barato, siempre disponible (incluso si H_k no es definida positiva), pero **conservador**: es esencialmente un paso de descenso de gradiente.

Paso de Newton p_N

$$p_N = -H_k^{-1} \nabla f_k$$

El minimizador **exacto** de m_k sin restricción de radio. Ideal cuando H_k es definida positiva y Δ_k es suficientemente grande — pero puede salirse de la región, o no ser descenso si H_k no es definida positiva.

¿De dónde sale la dirección de Cauchy?

El gradiente del modelo cuadrático respecto a p es

$$\nabla_p m_k(p) = \nabla f(x_k) + H_k\, p$$

Evaluado en $p = 0$ (el punto de partida x_k mismo, donde el término cuadrático se anula):

$$\nabla_p m_k(0) = \nabla f(x_k)$$

Por eso la dirección de Cauchy es $-\nabla f(x_k)$
La dirección de máximo descenso **del modelo cuadrático**, evaluada en el punto de partida, coincide exactamente con $-\nabla f(x_k)$. No es una casualidad ni una simplificación: es consecuencia directa de que m_k y f comparten el mismo gradiente en $p = 0$ (así se construye el modelo de Taylor).

Cuidado: minimizar *sobre una recta* no es minimizar el modelo completo

El paso de Cauchy se obtiene en dos etapas:

1. Fijar la dirección $d = -\nabla f(x_k)$ (la de máximo descenso de m_k en $p = 0$).
2. Minimizar m_k **restringido a esa recta**: $\min_{\alpha \geq 0} m_k(\alpha\, d)$.

Esto NO es lo mismo que minimizar m_k sobre todo $\mathbb{R}^n$
Si en cambio se minimiza $m_k(p)$ dejando que **dirección y magnitud** se ajusten libremente según la curvatura H_k, se llega al paso de **Newton**, $p_N = -H_k^{-1}\nabla f_k$ — que en general **no** es paralelo a $-\nabla f_k$.

Excepción: si $H_k = cI$ (curvatura isotrópica), $p_N = -\frac{1}{c}\nabla f_k$ sí es paralelo a p_C, y el dogleg colapsa a un segmento recto.

¿Qué pasa si H_k no es definida positiva?

Caso H_k definida positiva
$\nabla f_k^T H_k \nabla f_k > 0$: la parábola $m_k(\alpha\,(-\nabla f_k))$ abre hacia arriba, α_C es un mínimo genuino y bien definido.

Caso H_k NO definida positiva ($\nabla f_k^T H_k \nabla f_k \leq 0$)
La restricción de m_k a la recta $-\nabla f_k$ es cóncava o lineal: **no tiene mínimo finito**. Regla: ir hasta el borde de la región de confianza,

$$p_C = -\Delta_k \frac{\nabla f_k}{\|\nabla f_k\|}$$

Qué significa para el dogleg
El paso de Cauchy **siempre existe**: no invierte H_k y $-\nabla f_k$ siempre es dirección de descenso. El paso de Newton, en cambio, solo tiene sentido con H_k definida positiva; si no lo es, se usa **solo Cauchy** (o se cambia el solver del subproblema). Newton amortiguado, por su parte, necesita una salvaguarda externa.

La trayectoria dogleg

Se define la poligonal $p(\tau)$, $\tau \in [0, 2]$:

$$p(\tau) = \begin{cases} \tau\, p_C, & 0 \le \tau \le 1 \\ p_C + (\tau - 1)(p_N - p_C), & 1 \le \tau \le 2 \end{cases}$$

Es decir: primero va del origen a p_C en línea recta, y luego de p_C a p_N.

Propiedad clave (Nocedal & Wright, Lema 4.2)
$\|p(\tau)\|$ crece **monótonamente** con τ. Por lo tanto hay, a lo sumo, un único $\tau^\star$ en el segundo tramo donde la poligonal cruza la frontera $\|p\| = \Delta_k$ — y ese cruce se puede encontrar resolviendo una simple ecuación cuadrática en τ.

Regla de selección del paso dogleg

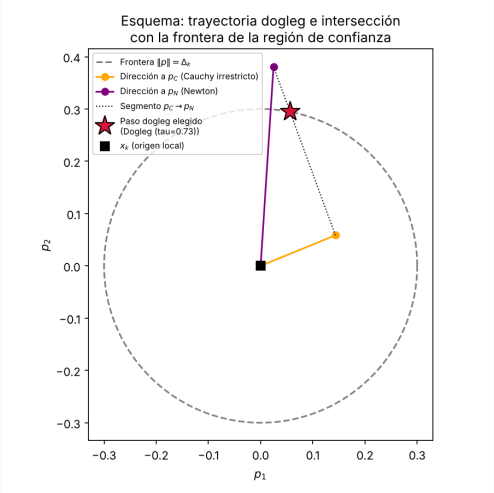
Dado el radio actual Δ_k:

1. **Si** $\|p_N\| \leq \Delta_k$: usar el paso de Newton completo, $p_k = p_N$ (cabe en la región; es el mejor paso posible).
2. **Si** $\|p_C\| \geq \Delta_k$: usar $p_k = \Delta_k \, p_C / \|p_C\|$ (ni el paso de Cauchy cabe; truncar a lo largo de $-\nabla f_k$).
3. **En otro caso** ($\|p_C\| < \Delta_k < \|p_N\|$): el paso cruza el segundo tramo. Resolver

$$\|p_C + \tau(p_N - p_C)\|^2 = \Delta_k^2, \qquad \tau \in [0, 1]$$

(ecuación cuadrática en τ, se toma la raíz positiva) y usar $p_k = p_C + \tau(p_N - p_C)$.

Esquema: cómo intersecta la poligonal la frontera



Cómo se ajusta el radio Δ_k en cada iteración

Se mide qué tan bien predijo el modelo cuadrático la reducción **real** de f:

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)} = \frac{\text{reducción real}}{\text{reducción predicha por el modelo}}$$

Regla de actualización

- $\rho_k < 1/4$ (predicción mala) $\Rightarrow$ **encoger**: $\Delta_{k+1} = 0{,}25\,\Delta_k$.
- $\rho_k > 3/4$ **y** $\|p_k\| = \Delta_k$ (el paso usó todo el radio disponible y el modelo fue fiel) $\Rightarrow$ **crecer**: $\Delta_{k+1} = \min(2\Delta_k,\ \Delta_{\max})$.
- En otro caso $\Rightarrow$ mantener $\Delta_{k+1} = \Delta_k$.

Aceptación del paso: si $\rho_k > 0$, se acepta $x_{k+1} = x_k + p_k$; si no, se rechaza ($x_{k+1} = x_k$) y solo se encoge la región para reintentar.

Código: paso dogleg (los 3 casos)

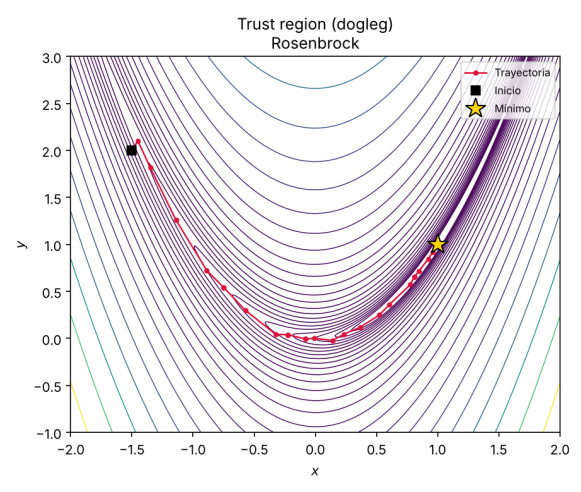
```
if np.linalg.norm(pN) <= Delta:
    p = pN                                    # caso 1: Newton completo
elif np.linalg.norm(pC) >= Delta:
    p = Delta / np.linalg.norm(pC) * pC        # caso 2: Cauchy truncado
else:
    d = pN - pC
    a = d @ d; b = 2*pC @ d; c = pC @ pC - Delta**2
    tau = (-b + np.sqrt(b**2 - 4*a*c)) / (2*a)  # caso 3: interseccion
    p = pC + tau * d
```

Código: actualización del radio (ratio ρ_k)

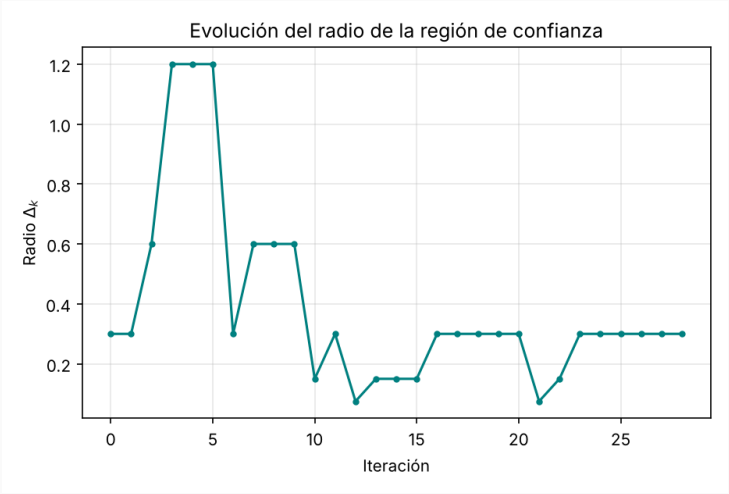
```
pred = -(g @ p + 0.5 * p @ H @ p)          # m(0) - m(p)
actual = f(x) - f(x + p)
rho = actual / pred if pred > 1e-14 else -1.0

if rho > 0:
    x = x + p                               # aceptar paso
if rho < 0.25:
    Delta = 0.25 * Delta                    # encoger
elif rho > 0.75 and np.isclose(np.linalg.norm(p), Delta):
    Delta = min(2 * Delta, Delta_max)       # crecer
```

Trayectoria: dogleg en Rosenbrock



Evolución del radio de la región de confianza



Resultados

- Trust region dogleg (manual): **29** iteraciones, mezcla de pasos "Newton" (cuando p_N cabe) y "Dogleg" (cuando cruza la frontera) — ver primeras 15 iteraciones: `[Newton, Dogleg(τ=0.06), Dogleg(τ=0.13), Newton, Newton, Newton, Dogleg(τ=0.46), ...]`
- Radio Δ_k: parte en 0,3, llega a oscilar entre 0,075 y 1,2 según qué tan bien predice el modelo local.
- `scipy.optimize.minimize(method='dogleg')`: 24 iteraciones — resultado muy similar, confirmando la implementación manual.

Por qué importa

Trust region es **robusto** porque el paso de Cauchy siempre existe y reduce el modelo: si H_k no es definida positiva se recurre a él (una línea en el código), sin la salvaguarda ad-hoc de Newton amortiguado.

Levenberg-Marquardt

Mínimos cuadrados no lineales

Un caso muy frecuente en ingeniería: ajustar un modelo a datos minimizando

$$f(x) = \tfrac{1}{2}\|r(x)\|^2 = \tfrac{1}{2}\sum_{i=1}^{m} r_i(x)^2, \qquad r : \mathbb{R}^n \to \mathbb{R}^m$$

donde $r_i(x)$ es el residuo (diferencia modelo–dato) del punto i.

Para este f en particular:

$$\nabla f(x) = J(x)^T r(x), \qquad H(x) = J(x)^T J(x) + \underbrace{\sum_i r_i(x)\,\nabla^2 r_i(x)}_{\text{se descarta}}$$

Aproximación de Gauss-Newton: $H(x) \approx J(x)^T J(x)$ (exacta si r es lineal, o si los residuos son pequeños cerca del óptimo).

El paso de Levenberg-Marquardt

Sistema regularizado

$$(J^T J + \lambda I)\, p = -J^T r$$

- $\lambda \to 0$: p tiende al paso de **Gauss-Newton** (casi-Newton, rápido si los residuos son pequeños).
- $\lambda \to \infty$: $p \to -\frac{1}{\lambda} J^T r$, un paso de **descenso de gradiente** muy corto (lento pero robusto).

Relación con trust region

λ juega el mismo papel que $1/\Delta_k$ en dogleg: se ajusta según el mismo ratio ρ_k de reducción real/predicha. Si el paso mejora f: aceptar y **bajar** λ (confiar más en Gauss-Newton). Si empeora: rechazar y **subir** λ (retroceder hacia gradiente). LM es, en esencia, un trust region implícito especializado a mínimos cuadrados.

Ejemplo: ajuste de un decaimiento exponencial

Datos sintéticos ruidosos generados con $y(t) = A\,e^{-kt} + b$, $(A, k, b) = (5,\ 0{,}7,\ 1)$, ruido gaussiano $\sigma = 0{,}15$. Estimación inicial deliberadamente mala: $(A_0, k_0, b_0) = (1,\ 0{,}1,\ 0)$.

$$r_i(A, k, b) = A\,e^{-kt_i} + b - y_i, \qquad i = 1, \ldots, 40$$

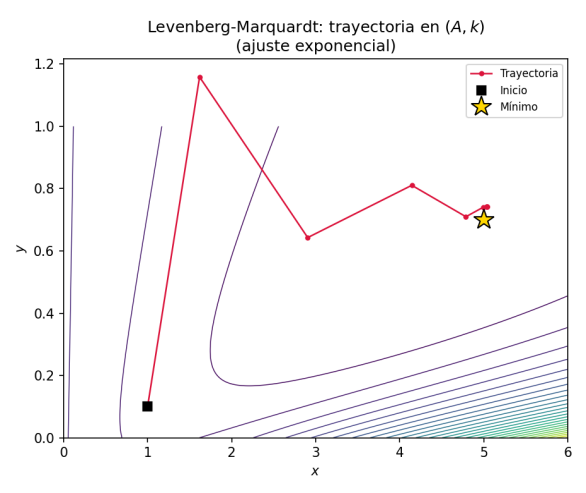
Jacobiano analítico: $\partial r_i/\partial A = e^{-kt_i}$, $\partial r_i/\partial k = -At_i e^{-kt_i}$, $\partial r_i/\partial b = 1$.

`scipy.optimize.curve_fit` y `scipy.optimize.least_squares(method='lm')` usan Levenberg-Marquardt por defecto para este tipo exacto de problema.

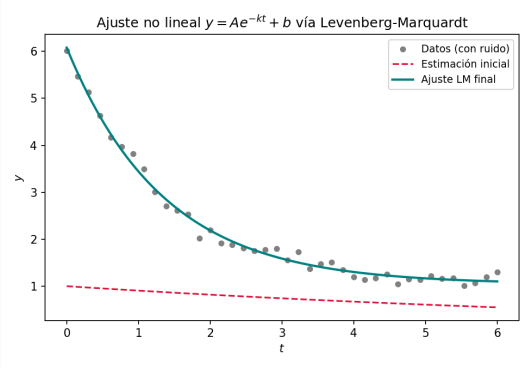
Código: núcleo de Levenberg-Marquardt

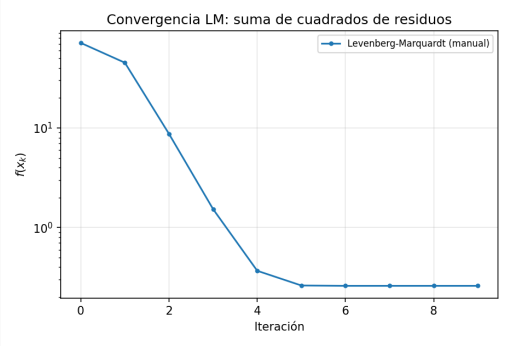
```
JTJ = J.T @ J;  g = J.T @ r;  f_cur = 0.5 * r @ r
for _ in range(30):
    A_lm = JTJ + lam * np.eye(n)
    p = np.linalg.solve(A_lm, -g)
    f_new = 0.5 * np.sum(res_fun(theta + p)**2)
    pred = -(g @ p + 0.5 * p @ JTJ @ p)
    rho = (f_cur - f_new) / pred if pred > 1e-14 else -1.0
    if rho > 0:
        theta = theta + p;  lam = max(lam*0.5, 1e-12);  break
    else:
        lam *= 3.0
```

Trayectoria en el espacio de parámetros (A, k)



Ajuste final y convergencia





- LM manual: **10** iteraciones, $(A, k, b)_{\text{final}} = (5{,}038,\ 0{,}741,\ 1{,}042)$.
- `scipy least_squares(lm)`: 10 evaluaciones, resultado **idéntico** hasta 5 cifras.
- Los valores no coinciden exactamente con (5, 0,7, 1) porque el ajuste es sobre datos **ruidosos** — eso es exactamente lo esperado en un problema de mínimos cuadrados.

Gradiente estocástico (SGD)

Motivación: cuando N es enorme

Muchos problemas (típicamente en aprendizaje automático, pero también en ajuste de modelos con muchísimos datos) tienen la forma

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x)$$

El gradiente completo $\nabla f(x) = \frac{1}{N} \sum_i \nabla f_i(x)$ cuesta $O(N)$ **por iteración**. Si N son millones de datos, eso es prohibitivo.

Idea: gradiente estocástico (SGD)
Reemplazar el gradiente completo por un **estimador ruidoso pero barato**, calculado sobre un subconjunto (*minibatch*) B_k de tamaño $b \ll N$ elegido al azar:

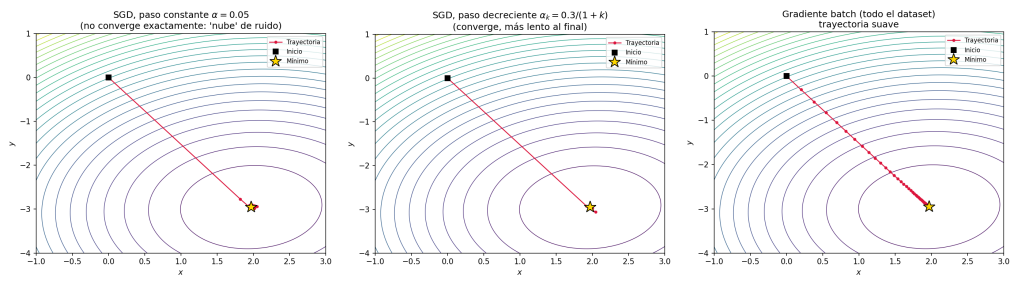
$$x_{k+1} = x_k - \alpha_k \cdot \frac{1}{b} \sum_{i \in B_k} \nabla f_i(x_k)$$

Paso constante vs. paso decreciente

Ejemplo: regresión lineal por mínimos cuadrados, $N = 500$ datos, minibatches de tamaño $b = 10$.

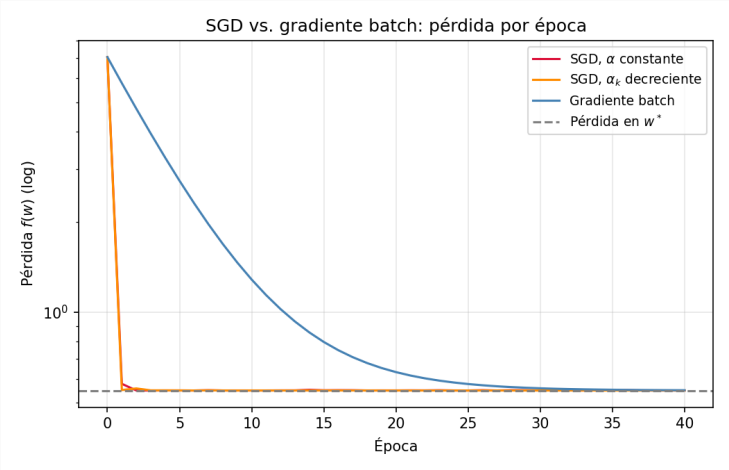
- **Paso constante** $\alpha = 0{,}05$: NO converge al óptimo exacto — la trayectoria entra en una "nube" de ruido alrededor de él (la varianza del estimador de gradiente no desaparece).
- **Paso decreciente** $\alpha_k = \alpha_0/(1 + k)$: sí converge, porque el ruido se apaga junto con el paso — a costa de avanzar más lento al final.

Trayectorias: paso constante vs. decreciente vs. batch



- $\|w - w^\star\|$ final: paso constante 0,0136, paso decreciente **0,0023**, batch 0,0454 (con el mismo número de épocas y un α razonable para cada uno).

Pérdida por época: SGD vs. gradiente batch



Por qué SGD baja más rápido *por época*

- En una época, SGD recorre los $N = 500$ datos — igual que un paso de gradiente batch — pero los agrupa en minibatches de $b = 10$ y **actualiza** w **50 veces** (una por minibatch), no una sola vez.
- Por eso, a igual número de épocas (mismo costo total de evaluaciones de ∇f_i), SGD hace muchas más actualizaciones de x y baja la pérdida mucho más rápido al principio.
- El precio: cada actualización individual es ruidosa (basada en solo 10 datos), de ahí la "nube" alrededor del óptimo con paso constante.

Conexión con el resto de la clase

SGD es descenso de gradiente (script 1) con un ∇f **aproximado**. Todo lo visto sobre el tamaño de paso sigue aplicando — salvo que ahora, además del tamaño, importa el **ruido** del estimador.

Síntesis

Tabla comparativa

Método	Usa	Estrategia	Convergencia
Grad. (paso fijo)	∇f	—	lineal, puede divergir
Grad. + Armijo	∇f	line search	lineal, robusta
Powell	solo f	line search (conjugadas)	superlineal
Newton	$\nabla f, H$	line search / amortiguado	cuadrática (local)
Quasi-Newton (BFGS)	∇f	line search	superlineal
Trust region (dogleg)	$\nabla f, H$	trust region	superlin./cuadrática
Levenberg-Marquardt	J (mín. cuadr.)	trust region implícito	superlineal
SGD	∇f_i (ruidoso)	line search implícito	sublineal (ruidosa)

- **Sin derivadas** $\rightarrow$ Powell. **Con gradiente, sin Hessiana** $\rightarrow$ BFGS. **Con Hessiana** $\rightarrow$ Newton o dogleg (dogleg más robusto si H no siempre es definida positiva). **Mínimos cuadrados** $\rightarrow$ Levenberg-Marquardt. **Datos masivos** $\rightarrow$ SGD.

Lo que tienen en común

- Todos generan una sucesión $x_{k+1} = x_k + \alpha_k d_k$ (o su equivalente en trust region, $x_{k+1} = x_k + p_k$).
- Todos necesitan algún mecanismo de **control de paso** (line search o radio de confianza) para garantizar que efectivamente se reduce f — sin él, como vimos en el paso fijo, se puede divergir.
- La **curvatura** (Hessiana exacta o aproximada) es lo que separa los métodos lentos (descenso de gradiente, decenas de miles de iteraciones en valles curvos) de los rápidos (Newton/quasi-Newton/dogleg, decenas de iteraciones).

Roadmap del curso

- **Clases 1–3**: fundamentos analíticos, búsqueda 1D/multidimensional, metaheurísticas estocásticas.
- **Clase 4 (hoy)**: métodos basados en dirección de búsqueda — gradiente, Newton, quasi-Newton, trust region, LM, SGD.
- **Próximos pasos posibles**: programación lineal (simplex, punto interior); programación no lineal con restricciones (SQP, métodos de penalización y de barrera, combinando las ideas de hoy con las condiciones KKT de la Clase 1).

Ejercicios propuestos

1. Repetir el descenso de gradiente de paso fijo variando κ (número de condición) y graficar el número de iteraciones necesarias vs. κ.
2. Implementar la condición de curvatura de Wolfe (además de Armijo) en el backtracking y comparar el número de iteraciones resultante en Rosenbrock.
3. Modificar Powell para reiniciar periódicamente a los ejes coordenados (cada n ciclos) y comparar la robustez con la versión original.
4. Comparar Newton puro (sin amortiguar) vs. Newton amortiguado partiendo de un punto donde la Hessiana de Rosenbrock no es definida positiva; verificar en qué iteraciones actúa la salvaguarda.
5. Implementar el método trust region con el subproblema resuelto exactamente (método de Moré-Sorensen) y comparar el número de iteraciones con dogleg.
6. Extender el ejemplo de SGD a un tamaño de minibatch variable ($b \in \{1, 10, 50, 500\}$) y estudiar el trade-off entre ruido y velocidad de convergencia por época.

Referencias

- J. Nocedal, S. J. Wright, *Numerical Optimization*, 2ª ed., Springer, 2006. (Cap. 3: line search; Cap. 4: trust region/dogleg; Cap. 6: quasi-Newton; Cap. 10: mínimos cuadrados no lineales.)
- S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- D. P. Bertsekas, *Nonlinear Programming*, 3ª ed., Athena Scientific, 2016.
- K. Levenberg, "A Method for the Solution of Certain Non-Linear Problems in Least Squares", *Quarterly of Applied Mathematics*, 1944.
- D. Marquardt, "An Algorithm for Least-Squares Estimation of Nonlinear Parameters", *SIAM J. Appl. Math.*, 1963.
- M. J. D. Powell, "An efficient method for finding the minimum of a function of several variables without calculating derivatives", *Computer Journal*, 1964.
- L. Bottou, F. E. Curtis, J. Nocedal, "Optimization Methods for Large-Scale Machine Learning", *SIAM Review*, 2018. (SGD.)

Fin de la Clase 4